

Example Selectors

Contents

- LengthBased ExampleSelector
- Similarity ExampleSelector
- Maximal Marginal Relevance ExampleSelector
- NGram Overlap ExampleSelector

If you have a large number of examples, you may need to select which ones to include in the prompt. The ExampleSelector is the class responsible for doing so. The base interface is defined as below.

```
class BaseExampleSelector(ABC):
    """Interface for selecting examples to include in prompts."""

    @abstractmethod
    def select_examples(self, input_variables: Dict[str, str]) ->
    List[dict]:
        """Select which examples to use based on the inputs."""
```

The only method it needs to expose is a `select_examples` method. This takes in the input variables and then returns a list of examples. It is up to each specific implementation as to how those examples are selected. Let's take a look at some below.

```
from langchain.prompts import FewShotPromptTemplate
```

LengthBased ExampleSelector

This ExampleSelector selects which examples to use based on length. This is useful when you are worried about constructing a prompt that will go over the length of the context window. For longer inputs, it will select fewer examples to include, while for shorter inputs it will select more.

 [latest](#) ▼

[Skip to main content](#)

```
from langchain.prompts import PromptTemplate
from langchain.prompts.example_selector import LengthBasedExampleSelector
```

```
# These are a lot of examples of a pretend task of creating antonyms.
examples = [
    {"input": "happy", "output": "sad"},
    {"input": "tall", "output": "short"},
    {"input": "energetic", "output": "lethargic"},
    {"input": "sunny", "output": "gloomy"},
    {"input": "windy", "output": "calm"},
]
```

```
example_prompt = PromptTemplate(
    input_variables=["input", "output"],
    template="Input: {input}\nOutput: {output}",
)
example_selector = LengthBasedExampleSelector(
    # These are the examples it has available to choose from.
    examples=examples,
    # This is the PromptTemplate being used to format the examples.
    example_prompt=example_prompt,
    # This is the maximum length that the formatted examples should be.
    # Length is measured by the get_text_length function below.
    max_length=25,
    # This is the function used to get the length of a string, which is
    # used
    # to determine which examples to include. It is commented out because
    # it is provided as a default value if none is specified.
    # get_text_length: Callable[[str], int] = lambda x:
    len(re.split("\n| ", x))
)
dynamic_prompt = FewShotPromptTemplate(
    # We provide an ExampleSelector instead of examples.
    example_selector=example_selector,
    example_prompt=example_prompt,
    prefix="Give the antonym of every input",
    suffix="Input: {adjective}\nOutput:",
    input_variables=["adjective"],
)
```

```
# An example with small input, so it selects all examples.
print(dynamic_prompt.format(adjective="big"))
```

Give the antonym of every input

Input: happy
Output: sad

Input: tall

  latest ▼

[Skip to main content](#)

Input: energetic
Output: lethargic

Input: sunny
Output: gloomy

Input: windy
Output: calm

Input: big
Output:

```
# An example with long input, so it selects only one example.  
long_string = "big and huge and massive and large and gigantic and tall  
and much much much much much bigger than everything else"  
print(dynamic_prompt.format(adjective=long_string))
```

Give the antonym of every input

Input: happy
Output: sad

Input: big and huge and massive and large and gigantic and tall and much
much much much much bigger than everything else
Output:

```
# You can add an example to an example selector as well.  
new_example = {"input": "big", "output": "small"}  
dynamic_prompt.example_selector.add_example(new_example)  
print(dynamic_prompt.format(adjective="enthusiastic"))
```

Give the antonym of every input

Input: happy
Output: sad

Input: tall
Output: short

Input: energetic
Output: lethargic

Input: sunny
Output: gloomy

Input: windy
Output: calm

Input: big
Output: small

  latest ▼

[Skip to main content](#)

Input: enthusiastic
Output:

Similarity ExampleSelector

The SemanticSimilarityExampleSelector selects examples based on which examples are most similar to the inputs. It does this by finding the examples with the embeddings that have the greatest cosine similarity with the inputs.

```
from langchain.prompts.example_selector import  
SemanticSimilarityExampleSelector  
from langchain.vectorstores import Chroma  
from langchain.embeddings import OpenAIEmbeddings
```

```
example_selector = SemanticSimilarityExampleSelector.from_examples(  
    # This is the list of examples available to select from.  
    examples,  
    # This is the embedding class used to produce embeddings which are  
    used to measure semantic similarity.  
    OpenAIEmbeddings(),  
    # This is the VectorStore class that is used to store the embeddings  
    and do a similarity search over.  
    Chroma,  
    # This is the number of examples to produce.  
    k=1  
)  
similar_prompt = FewShotPromptTemplate(  
    # We provide an ExampleSelector instead of examples.  
    example_selector=example_selector,  
    example_prompt=example_prompt,  
    prefix="Give the antonym of every input",  
    suffix="Input: {adjective}\nOutput:",  
    input_variables=["adjective"],  
)
```

Running Chroma using direct local API.
Using DuckDB in-memory for database. Data will be transient.

```
# Input is a feeling, so should select the happy/sad example  
print(similar_prompt.format(adjective="worried"))
```

Give the antonym of every input

Input: happy

 [latest](#) ▼

[Skip to main content](#)

Input: worried
Output:

```
# Input is a measurement, so should select the tall/short example
print(similar_prompt.format(adjective="fat"))
```

Give the antonym of every input

Input: happy
Output: sad

Input: fat
Output:

```
# You can add new examples to the SemanticSimilarityExampleSelector as
well
similar_prompt.example_selector.add_example({"input": "enthusiastic",
"output": "apathetic"})
print(similar_prompt.format(adjective="joyful"))
```

Give the antonym of every input

Input: happy
Output: sad

Input: joyful
Output:

Maximal Marginal Relevance ExampleSelector

The MaxMarginalRelevanceExampleSelector selects examples based on a combination of which examples are most similar to the inputs, while also optimizing for diversity. It does this by finding the examples with the embeddings that have the greatest cosine similarity with the inputs, and then iteratively adding them while penalizing them for closeness to already selected examples.

```
from langchain.prompts.example_selector import
MaxMarginalRelevanceExampleSelector
from langchain.vectorstores import FAISS
```

```
example_selector = MaxMarginalRelevanceExampleSelector.from_examples(
      latest ▼
```

[Skip to main content](#)

```

    # This is the embedding class used to produce embeddings which are
    used to measure semantic similarity.
    OpenAIEmbeddings(),
    # This is the VectorStore class that is used to store the embeddings
    and do a similarity search over.
    FAISS,
    # This is the number of examples to produce.
    k=2
)
mmr_prompt = FewShotPromptTemplate(
    # We provide an ExampleSelector instead of examples.
    example_selector=example_selector,
    example_prompt=example_prompt,
    prefix="Give the antonym of every input",
    suffix="Input: {adjective}\nOutput:",
    input_variables=["adjective"],
)

```

```

# Input is a feeling, so should select the happy/sad example as the
first one
print(mmr_prompt.format(adjective="worried"))

```

Give the antonym of every input

Input: happy
Output: sad

Input: windy
Output: calm

Input: worried
Output:

```

# Let's compare this to what we would just get if we went solely off of
similarity
similar_prompt.example_selector.k = 2
print(similar_prompt.format(adjective="worried"))

```

Give the antonym of every input

Input: enthusiastic
Output: apathetic

Input: worried
Output:

NGram Overlap ExampleSelector

The NGramOverlapExampleSelector selects and orders examples based on which examples are most similar to the input, according to an ngram overlap score. The ngram overlap score is a float between 0.0 and 1.0, inclusive.

The selector allows for a threshold score to be set. Examples with an ngram overlap score less than or equal to the threshold are excluded. The threshold is set to -1.0, by default, so will not exclude any examples, only reorder them. Setting the threshold to 0.0 will exclude examples that have no ngram overlaps with the input.

```
from langchain.prompts import PromptTemplate
from langchain.prompts.example_selector.ngram_overlap import
NGramOverlapExampleSelector
```

```
# These are examples of a fictional translation task.
examples = [
    {"input": "See Spot run.", "output": "Ver correr a Spot."},
    {"input": "My dog barks.", "output": "Mi perro ladra."},
    {"input": "Spot can run.", "output": "Spot puede correr."},
]
```

```
example_prompt = PromptTemplate(
    input_variables=["input", "output"],
    template="Input: {input}\nOutput: {output}",
)
example_selector = NGramOverlapExampleSelector(
    # These are the examples it has available to choose from.
    examples=examples,
    # This is the PromptTemplate being used to format the examples.
    example_prompt=example_prompt,
    # This is the threshold, at which selector stops.
    # It is set to -1.0 by default.
    threshold=-1.0,
    # For negative threshold:
    # Selector sorts examples by ngram overlap score, and excludes none.
    # For threshold greater than 1.0:
    # Selector excludes all examples, and returns an empty list.
    # For threshold equal to 0.0:
    # Selector sorts examples by ngram overlap score,
    # and excludes those with no ngram overlap with input.
)
dynamic_prompt = FewShotPromptTemplate(
    # We provide an ExampleSelector instead of examples.
    example_selector=example_selector,
    example_prompt=example_prompt,
    prefix="Give the Spanish translation of every input",
    suffix="Input: {sentence}\nOutput:",

```

 [latest](#) ▼

[Skip to main content](#)

```
    input_variables=["sentence"],  
)
```

```
# An example input with large ngram overlap with "Spot can run."  
# and no overlap with "My dog barks."  
print(dynamic_prompt.format(sentence="Spot can run fast."))
```

Give the Spanish translation of every input

Input: Spot can run.
Output: Spot puede correr.

Input: See Spot run.
Output: Ver correr a Spot.

Input: My dog barks.
Output: Mi perro ladra.

Input: Spot can run fast.
Output:

```
# You can add examples to NGramOverlapExampleSelector as well.  
new_example = {"input": "Spot plays fetch.", "output": "Spot juega a  
buscar."}  
  
example_selector.add_example(new_example)  
print(dynamic_prompt.format(sentence="Spot can run fast."))
```

Give the Spanish translation of every input

Input: Spot can run.
Output: Spot puede correr.

Input: See Spot run.
Output: Ver correr a Spot.

Input: Spot plays fetch.
Output: Spot juega a buscar.

Input: My dog barks.
Output: Mi perro ladra.

Input: Spot can run fast.
Output:

```
# You can set a threshold at which examples are excluded.  
# For example, setting threshold equal to 0.0  
# excludes examples with no ngram overlaps with input.
```

 [latest](#) ▼

[Skip to main content](#)


```
example_selector.threshold=0.0
print(dynamic_prompt.format(sentence="Spot can run fast."))
```

Give the Spanish translation of every input

Input: Spot can run.

Output: Spot puede correr.

Input: See Spot run.

Output: Ver correr a Spot.

Input: Spot plays fetch.

Output: Spot juega a buscar.

Input: Spot can run fast.

Output:

```
# Setting small nonzero threshold
example_selector.threshold=0.09
print(dynamic_prompt.format(sentence="Spot can play fetch."))
```

Give the Spanish translation of every input

Input: Spot can run.

Output: Spot puede correr.

Input: Spot plays fetch.

Output: Spot juega a buscar.

Input: Spot can play fetch.

Output:

```
# Setting threshold greater than 1.0
example_selector.threshold=1.0+1e-9
print(dynamic_prompt.format(sentence="Spot can play fetch."))
```

Give the Spanish translation of every input

Input: Spot can play fetch.

Output: